Introduction

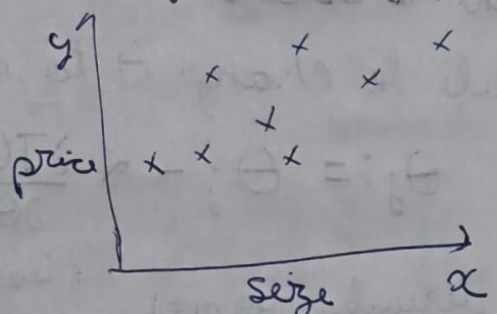
- machine learning
  - supervised
    - regression
    - classification
  - unsupervised → clustering

Supervised Learning -
when we have (x, y) ie labelled data and we essentially need to find a map

Unsupervised Learning -
when we only have x ie unlabelled data
Ex clustering → used in google news

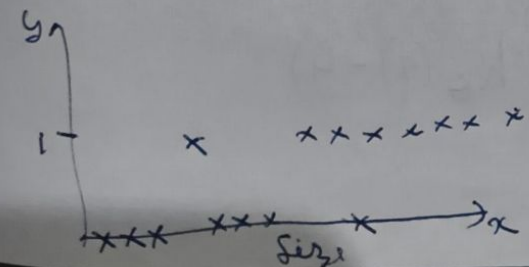
Regression -
y axis has continous values / real numbers
x axis can ofc be of multiple dimensions

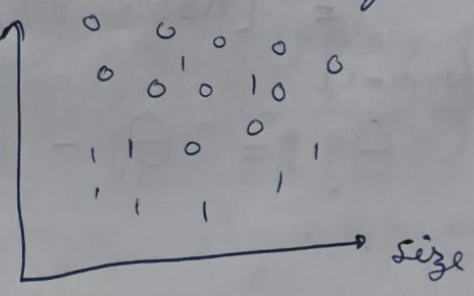
Ex house pricing



Classification
not continous, instead has classes, like 2 or more

Ex cancer type 0 → good 1 → bad nothing else





Linear Regression

Structure Definition

- Linear regression is a supervised learning regression algorithm.
- We create hypothesis & minimise the loss

$$h(\overset{x}{\theta}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \dots \theta_n x_n$$

$$\Rightarrow h = \theta^T x \text{ where } \theta^T = [\theta_0\ \theta_1 \dots],\ x = [1\ x_1\ x_2 \dots]$$

- notation : n = no of training examples
  n = dimension of x
  x = input / features
  y = output / target variable (one dimensional)

- loss = $J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^i) - y^i)^2$

Gradient Descent

update rule to change θ to minimize $J(\theta)$

for each $\theta_j \rightarrow \theta_j := \theta_j - \alpha \frac{\partial J(\theta_j)}{\partial \theta_j}$

↳ learning rate usually 0.01 in practice

↓ doing derivation for $n=1$

$$\frac{\partial J(\theta_j)}{\partial \theta_j} = \left(\frac{1}{2}\right)(2)\left[h_\theta(x) - y\right] \frac{\partial}{\partial \theta_j}\left[\theta_0 + \theta_1 x_1 \dots \theta_n x_n\right]$$

$$= [h_\theta(x) - y][x_j]$$

(still for $n=1$).

$$\Rightarrow \theta_j := \theta_j - (\alpha \cdot x_j)(h_\theta(x) - y)$$

Batch Gradient Descent

$\theta_j := \theta_j - (\alpha \cdot x_j)\left(\sum_{i=1}^{m} [h_\theta(x^i) - y^i]\right)$

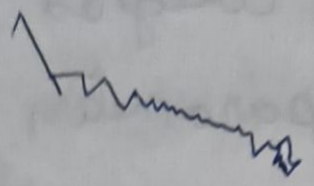
↳ size of one batch

- slow
- converges to minima

Stochastic Gradient Descent

for $(i=1;\ i<=m;\ i++)\ \{\theta_j := \theta_j - (\alpha x_j)(h_\theta(x^i) - y^i)\}$

- faster
- does not converg



Normal Equation

- works only for linear regression

$X \rightarrow n \times 1$

$X^T \rightarrow 1 \times n$

$Y \rightarrow 1 \times 1$

↳ $\theta_{optimal} = (X^T X)^{-1} X^T Y$

notation: $\nabla_\theta J\theta = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \end{bmatrix}$, if $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial a_{11}} & \frac{\partial f}{\partial a_{12}} \\ \frac{\partial f}{\partial a_{21}} & \frac{\partial f}{\partial a_{22}} \end{bmatrix}$$

- some properties: if $f(A) = tr(AB)$ then $\nabla_A f(A) = B^T$

$\nabla_A tr(AA^T C) = CA + C^T A$

↳ similar to $\frac{\partial a^2 c}{\partial a} = 2ac$

we define $X\theta = \begin{bmatrix} h_\theta(x^1) \\ h_\theta(x^2) \\ \vdots \\ h_\theta(x^m) \end{bmatrix}$ $y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y^m \end{bmatrix}$ ∵ $z^T z = \sum z_i^2$

now we can re-write $J(\theta) = \frac{1}{2}(X\theta - y)^T (X\theta - y)$

now $\nabla_\theta J(\theta) = \nabla_\theta \frac{(\theta^T X^T - y^T)(X\theta - y)}{2} = \frac{1}{2}\nabla_\theta (\theta^T X^T X \theta - y^T X\theta - \theta^T X^T y + y^T y)$

using property: $= \frac{1}{2}(X^T X\theta + X^T X\theta - X^T y - X^T y)$

$= X^T X\theta - X^T y$ is to be set 0

$\Rightarrow \theta = (X^T X)^{-1} X^T y$

→ $\theta = (X^T X)^{-1} X^T y$

Locally Weighted Regression

this algo gives more importance to local data for that we use a weight function

- locally weighted regression is <u>non parametric</u>

ie no. of parameters grow (likely) with more data

in LWR:

find θ to minimize $\sum_{i=1}^{n} w^{(i)} \left(y^{(i)} - \theta^T x^{(i)}\right)^2$

where $w^{(i)}$ is weight function, mostly:

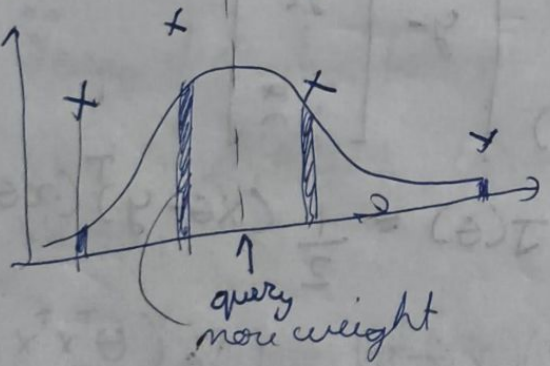
$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

$\tau \rightarrow$ changes width of $w^{(i)}$

$x \rightarrow$ query point.
ie where we need to use this model

$\rightarrow$ $|x^{(i)} - x|$ is small $\Rightarrow$ closer $\Rightarrow$ large weight
$|x^{(i)} - x|$ is big $\Rightarrow$ farther $\Rightarrow$ small weight

Ex:



Why we use least squares in ~~logistic~~ linear regression

(★)

$y^{(i)}$ will not be exactly same as $\theta^T x^{(i)}$ so that "error" / noise will be ε.

we don't give ε a direct value, instead represent it as a distribution, so

$y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$ where $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$

↓ distributed as; ↳ normal/gaussian distribution

$$P(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\varepsilon^{(i)2}}{2\sigma^2}\right)$$ → integrates to 1

Assumption: $\varepsilon^{(i)}$ are IID (independent & identically distributed)

$$\Rightarrow P(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

ie $P(y^{(i)} | x^{(i)}; \theta) \sim \mathcal{N}(\theta^T x^{(i)} | \sigma^2)$

probability of data, likelyhood of params

$$L(\theta) = P(\vec{y} | x; \theta) = \prod_{i=1}^{n} P(y^{(i)} | x^{(i)}; \theta)$$

over all n

$$= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

so we need to maximize likelyhood. ie let $l(\theta) = \log(L(\theta))$, we maximize l instead

~~maximize~~ so minimize $\sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2 = 2J(\theta)$

hera proved.

Classification

Logistic Regression

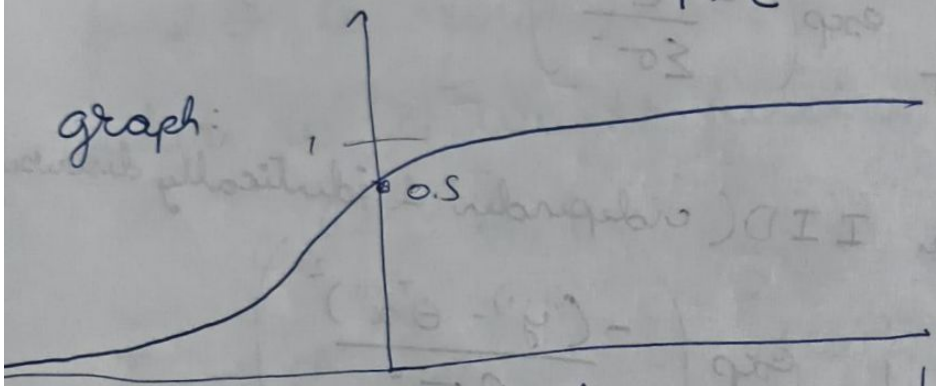
#Definition -

we want to compress the output of hypothesis b/w 0 and 1,

we use segmoid : $\frac{1}{1+e^{-z}} = g(z)$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$$

graph:



$\theta^T x$ just like linear reg. but we use segmoid to get ans.: $P(y|x;\theta) = (h_\theta(x^i))^{y^{(i)}} \cdot (1-h_\theta(x^i))^{1-y^{(i)}}$

θ update rule:-

$$L(\theta) = P(\vec{y}|x;\theta) = \prod_{i=1}^{m} P(y^{(i)}|x^{(i)};\theta)$$

$$= \prod_{i=1}^{m} (h_\theta(x^{(i)}))^{y^{(i)}} (1-h_\theta(x^{(i)}))^{(1-y^{(i)})}$$

so to maximize likelyhood, maximize $\log(L)$ instead $= l$

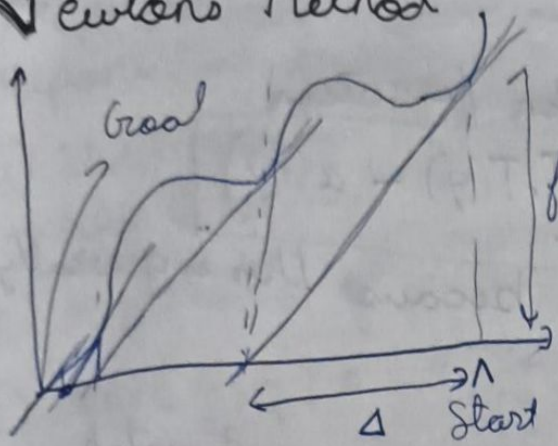
$$l(\theta) = \sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(x^i) + (1-y)\left(\log(1-h_\theta(x^i))\right)\right]$$

now we change θ to maximize the

$\theta := \theta \pm \frac{\partial l}{\partial \theta}$ → + when maximize l / − when minimize J ↑ Same as linear reg

after derivation we get: $\frac{\partial l}{\partial \theta_j} = \left[\sum_{i=1}^{m} (y^i - h_\theta(x_i))^2\right] x_j$

Newton's Method



we know $f'(\theta) = \frac{f(\theta)}{\Delta}$

$\Rightarrow \Delta = \frac{f(\theta)}{f'(\theta)}$

hence we get $\theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^t)}{f'(\theta^t)}$

- newton's method is here used to reach 0 fast
- how we apply it is: we put $f(\theta) = l'(\theta)$ and get θ for max log likelyhood

$$\theta^{(t+1)} = \theta^{(t)} - \frac{l'(\theta^t)}{l''(\theta^t)}$$

- newton's method has quadratic convergence i.e error : 0.01 → 0.0001 → 0.00000001

Newton Method for vector θ (*)

$$\theta^{t+1} := \theta^t + (H_\theta^{-1} \nabla_\theta(l))$$

l' is vector in $\mathbb{R}^{n+1}$

$H \rightarrow$ hessian matrix, vector $\mathbb{R}^{n+1} \times \mathbb{R}^{n+1}$

$$\nabla_\theta l = \begin{bmatrix} \frac{\partial l}{\partial \theta_1} \\ \frac{\partial l}{\partial \theta_1} \\ \vdots \end{bmatrix}$$

$$H_{ij} = \frac{\partial^2 l}{\partial \theta_i \partial \theta_j}$$

hence newton's method is not suitable if θ vector has more than a few thousands of features

---

Perceptron Algo

let $g(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$

$h_\theta(x) = g(\theta^T x)$ in this algo.

$y^i - h_\theta(x^i) = 0$ if correct guess, $+/-1$ if wrong guess

→ same update rule : $\theta_j := \theta_j + (\alpha x_j^i)(y^i - h_\theta(x^i))$

Exponential Families

- pdf - probability distribution function

$$p(y;\eta) = b(y) \cdot \exp(\eta^T \cdot T(y) - a(\eta))$$

- $y \equiv$ data we don't use x because this is generally used as output of an algo
- $\eta \equiv$ natural parameter $\geq$ dimensions must match
- $T(y) \equiv$ sufficient statistic (mostly y in this lecture)
- $b(y) \equiv$ base measure
- $a(\eta) \equiv$ log partition function

Bernoulli Distribution → for classification (logistic reg)

ϕ: Probability of event

$$p(y^i;\phi) = \phi^{y^i} \cdot (1-\phi)^{(1-y^i)} \rightarrow \text{pdf of bernoulli}$$

now we want to represent this pdf as expo family

$$p(y;\phi) = \exp\left(y^i \log(\phi) + (1-y^i)\log(1-\phi) \right) \quad \Big| \quad \because \phi = \frac{1}{1+e^{-\eta}}$$

$$= 1 \cdot \exp\left(y^i \log\left(\frac{\phi}{1-\phi}\right) - \log(1-\phi) \right)$$

$b(y)$ $T(y)$ η $a(\eta) = \log(1+e^{\eta})$

Gaussian Distribution → for regression (fixed variance) Assume $\sigma^2 = 1$

$$\text{PDF} \rightarrow p(y^{(i)};\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(\mu - y^{(i)})^2}{2} \right)$$

$$\rightarrow p(y^{(i)};\mu) = \underbrace{\frac{e^{-\frac{y^2}{2}}}{\sqrt{2\pi}}}_{b(y)} \exp\left(\underbrace{\mu}_{\eta} \underbrace{y}_{T(y)} - \underbrace{\frac{1}{2}\mu^2}_{a(\eta)} \right)$$

Bernoulli → logistic reg
Gaussian → linear reg

Count → Poisson
Real Value → Gaussian
Binary → Bernoulli

$\mathbb{R}^+ \rightarrow$ Gamma Distribution ↓ Beta, Bayesian

Properties

exponential families have some useful props:

ⓐ MLE (maximum likelihood) wrt η is concave
⇒ NLL (negative log ·) is convex

ⓑ expectation of y $E[y;\eta] = \frac{\partial a(\eta)}{\partial \eta}$

ⓒ $Var[y;\eta] = \frac{\partial^2 a(\eta)}{\partial^2 \eta}$

} H.W prove them

General Linear Models (GLM)

Design Choices / Assumptions -

- $y|x;\theta \sim$ exponential family (η)
  - ↑ is a number
- $\eta = \theta^T x$, $h_\theta(x) = E[y;\eta] = E[y;\theta^T x]$

#Blueprint: $x \rightarrow$ [linear model $\theta^T x$] $\rightarrow \eta \rightarrow$ end form is expo family with a, b, T,

so we train θ so that it can give $\theta^T x$ which is η the params η is plugged in final ans, and that is the hypothesis, so during training maximize $\log(P(y|\theta^T x))$ Σ if batch,

Training GLM.

update rule stays same. $\theta_j := \theta_j + \alpha x_j (y_i - h_\theta(x_i))$

$E[y;\eta] = g(\eta) \rightarrow$ canonical response func^n. link function

$\eta = g^{-1}(\mu)$ ← mean of $E[y;\eta]$

and as per properties $g(\eta) = \frac{\partial a(\eta)}{\partial \eta}$ ⌈ the only learnable

- we have 3 paras → model para (θ)
  - ↓ $\eta = \theta^T x$
  - → natural para (η)
    - ↓ g, ↑ g^{-1}
  - → canonical para (ϕ) for bernoulli, (μ, σ^2) for gaussian

Eg: in logistic regression $h_\theta(x) = E[y|x;\theta] = \phi = \frac{1}{1+e^{-\eta}}$

Softmax Regression (*) → cross entropy minimisation

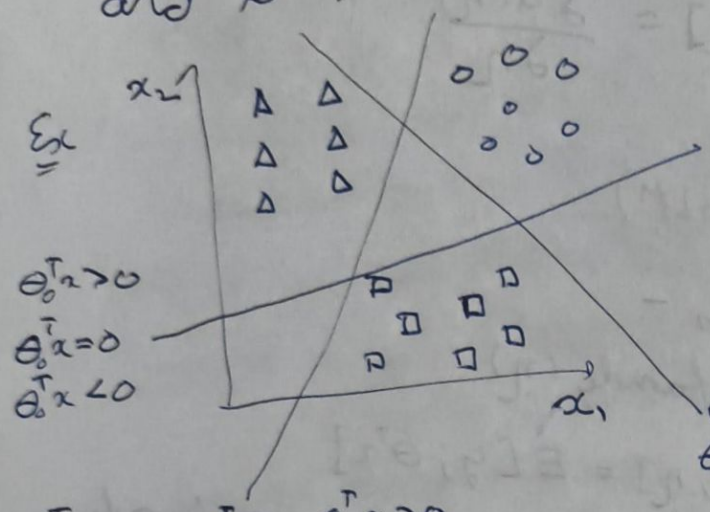
Say we have 3 classes, we want model to input x and tell which class it belongs to

let $k \to$ no. of classes

$x^{(i)} \in \mathbb{R}^n$

Label: $y \in \{[0,1]^k\}$

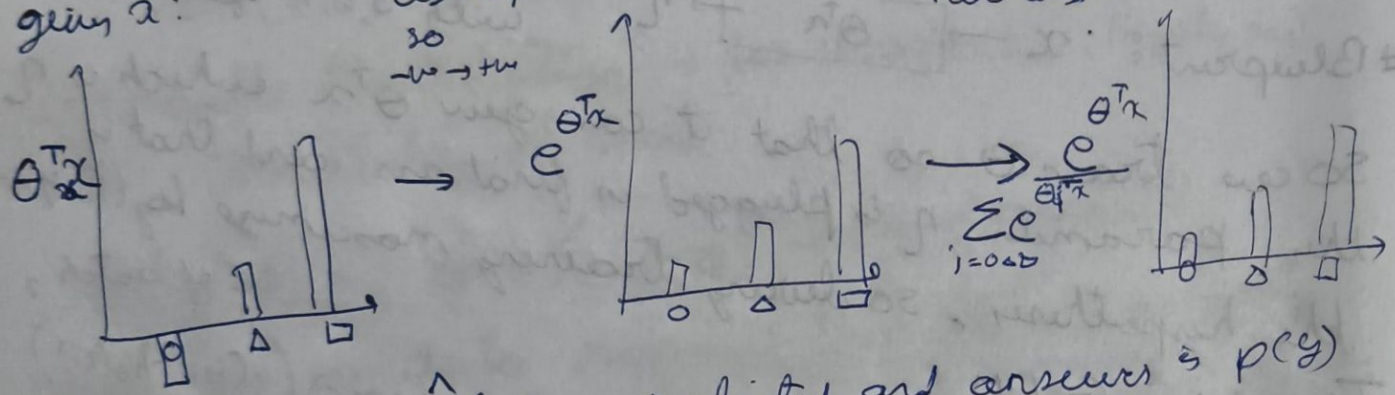
ex: $[0,0,1]$ → one hot vector



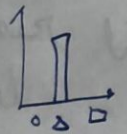
given x:

use exp so $-\infty \to +\infty$

normalise



now this is $\hat{p}(y)$ → predicted and answers is $p(y)$

maybe $p(y)$:  , so we calculate error and reduce it ∴ $p_\square = 0$, $p_\circ = 0$, $p_\Delta = 1$

$$\text{Cross ent}(\hat{p}, p) = -\sum_{y \in (\circ,\Delta,\square)} p(y) \cdot \log(\hat{p}(y)) = -\log(\hat{p}_\Delta(y))$$

$$= -\log\left(\frac{e^{\theta_\Delta^T x}}{\sum_{i=\Delta\square\circ} e^{\theta_i^T x}}\right)$$